UVM Verification of a

UART Core

Architecture of The Verification Environment

by

Hossam Shaher Anis

Cairo, 2026

Contents

Architecture of The Verification Environment 1
Contents 2
List of Figures 3
1 Introduction 4
2 The Top-Level Module 4
3 The Environment 4
4 Agents 6
4.1 uart_agent_system 7
4.2 uart_agent_rx 8
4.3 uart_agent_tx 9
5 Sequence Items 10
5.1 System Sequence Item 10
5.2 Rx Sequence Items 10
5.3 Tx Sequence Items 11
6 The Scoreboard 12
7 Sequences and Tests 13
7.1 Sequences 13
7.2 Tests 13

List of Figures

Figure 1 Structure of the top-level module `tb` 4
Figure 2 Structure of the environment `m_uart_env` (transaction-level communication is not shown) 5
Figure 3 Structure of the environment `m_uart_env` (transaction-level communication is shown) 5
Figure 4 UML class diagram of `uart_env` 6
Figure 5 `uart_agent_system`, `uart_agent_rx`, and `uart_agent_tx` extend `uvm_agent` 6
Figure 6 Structure of `m_uart_agent_system` 7
Figure 7 UML class diagram of `uart_agent_system` 7
Figure 8 Structure of `m_uart_agent_rx` 8
Figure 9 UML class diagram of `uart_agent_rx` 8
Figure 10 Structure of `m_uart_agent_tx` 9
Figure 11 UML class diagram of `uart_agent_tx` 9
Figure 12 UML class diagram of `uart_seq_item_system` 10
Figure 13 UML class diagrams of `uart_seq_item_*_rx` 10
Figure 14 UML class diagrams of `uart_seq_item_*_tx` 11
Figure 15 Structure of `m_uart_scoreboard` 12
Figure 16 UML class diagram of `uart_scoreboard` 12
Figure 17 UML class diagrams of `uart_sequence_*` and `uart_virt_sequence_*` 14
Figure 18 UML class diagrams of `uart_test_*` 15

1 Introduction

Serial communication uses a single data line (***serial line***) to exchange information between two systems.

The transmitting system converts the parallel data to a serial stream, and the receiving system reassembles the serial data back to its original parallel format.

A **UART (Universal Asynchronous Receiver and Transmitter) core** is the most commonly used scheme.

In this project, the UART core is verified using the SystemVerilog HDVL and the Universal Verification Methodology (UVM).

This document shows the architecture of the verification environment, the transaction-level communication between its UVM components, and UML class diagrams of some classes.

2 The Top-Level Module

Figure 1 shows the structure of the top-level module, `tb`.

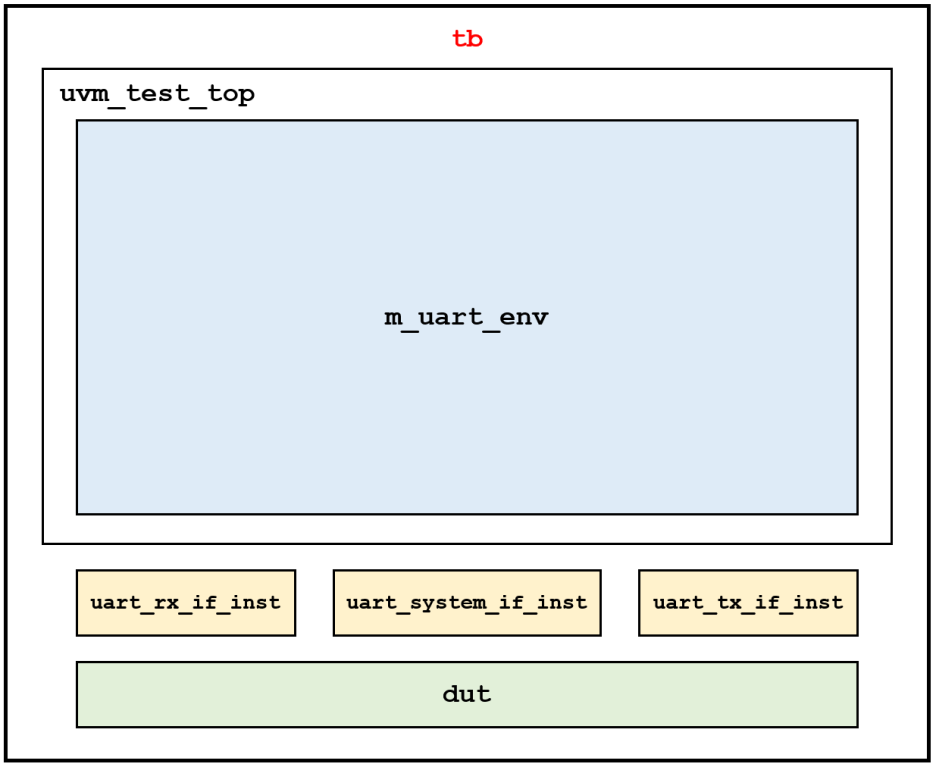

Figure 1 Structure of the top-level module `tb`

3 The Environment

Figure 2 shows the structure of the environment, `m_uart_env`, which is an instance of `uart_env`. (Transaction-level communication between its sub-components is omitted for clarity).

Figure 3 shows the structure of the environment with the transaction-level communication between its sub-components.[1]

Figure 4 shows the UML class diagram of `uart_env` which extends `uvm_env`.

[1] In this document, names of TLM ports are in **green**.

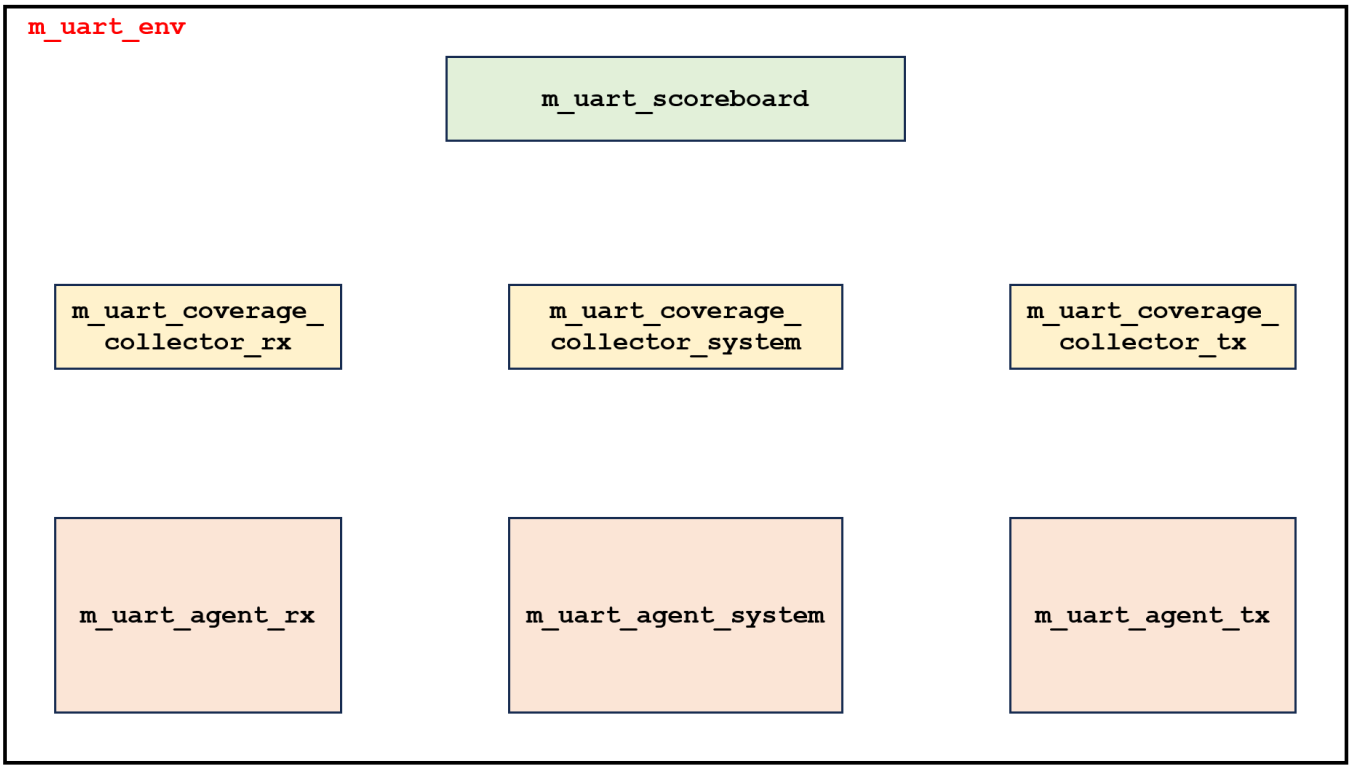

Figure 2 Structure of the environment `m_uart_env` (transaction-level communication is not shown)

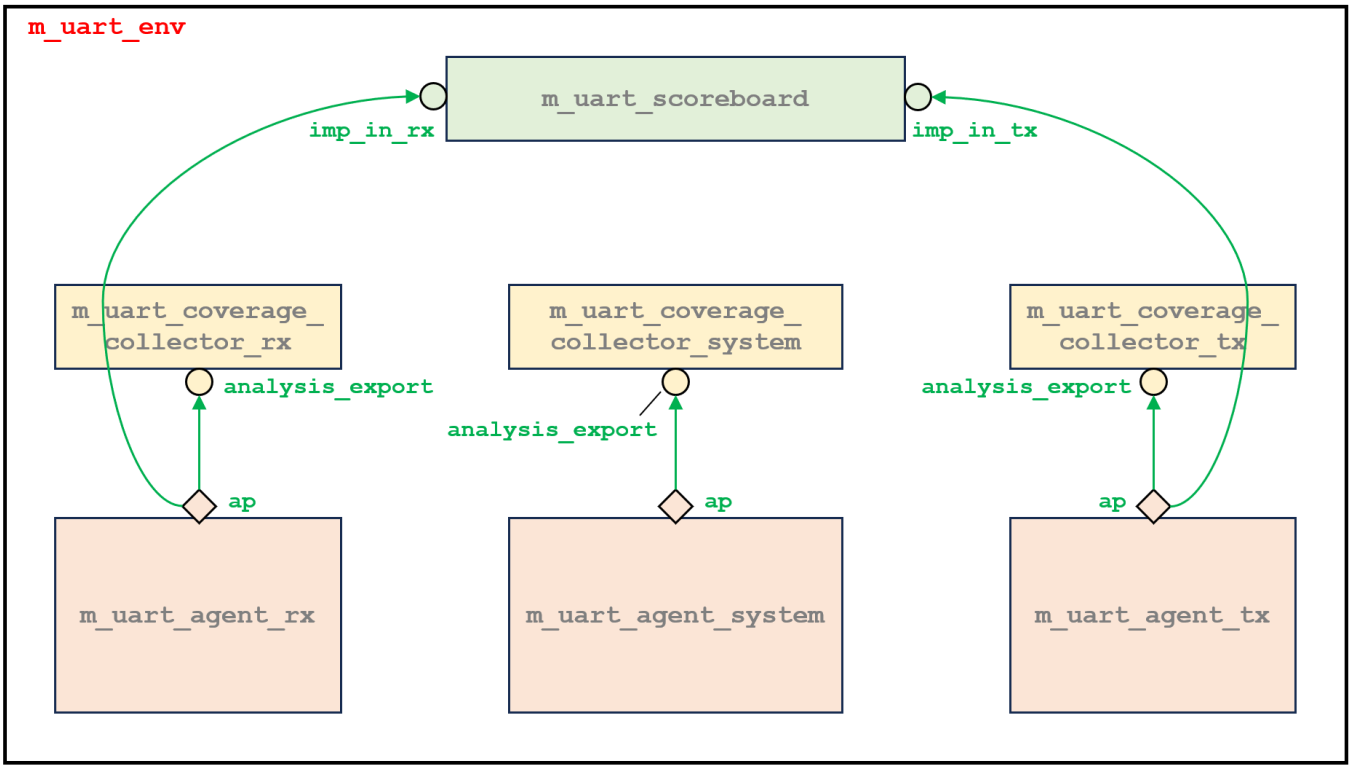

Figure 3 Structure of the environment `m_uart_env` (transaction-level communication is shown)

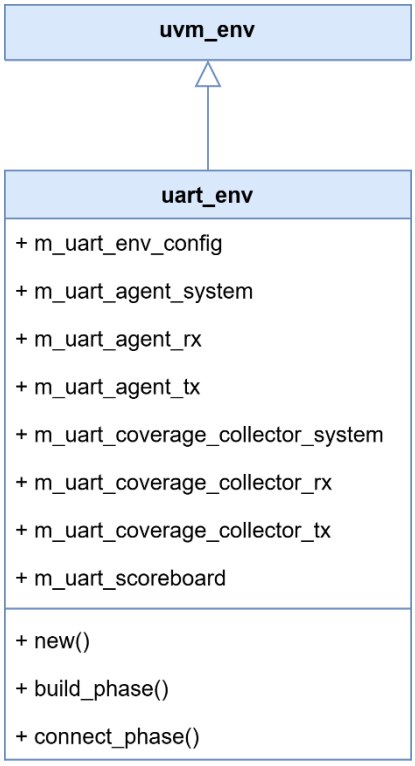

Figure 4 UML class diagram of `uart_env`

4 Agents

There are three agents in our environment:

- `m_uart_agent_system`, which is an instance of `uart_agent_system`.
- `m_uart_agent_rx`, which is an instance of `uart_agent_rx`.
- `m_uart_agent_tx`, which is an instance of `uart_agent_tx`.

These three agents extend `uvm_agent`. See Figure 5.

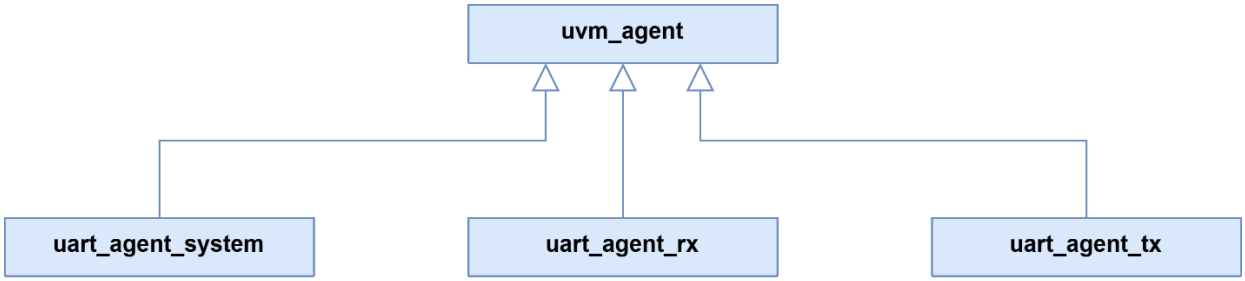

Figure 5 `uart_agent_system`, `uart_agent_rx`, and `uart_agent_tx` extend `uvm_agent`

4.1 uart_agent_system

Figure 6 shows the structure of `m_uart_agent_system`, which is an instance of `uart_agent_system`.

Figure 7 shows the UML class diagram of `uart_agent_system`.

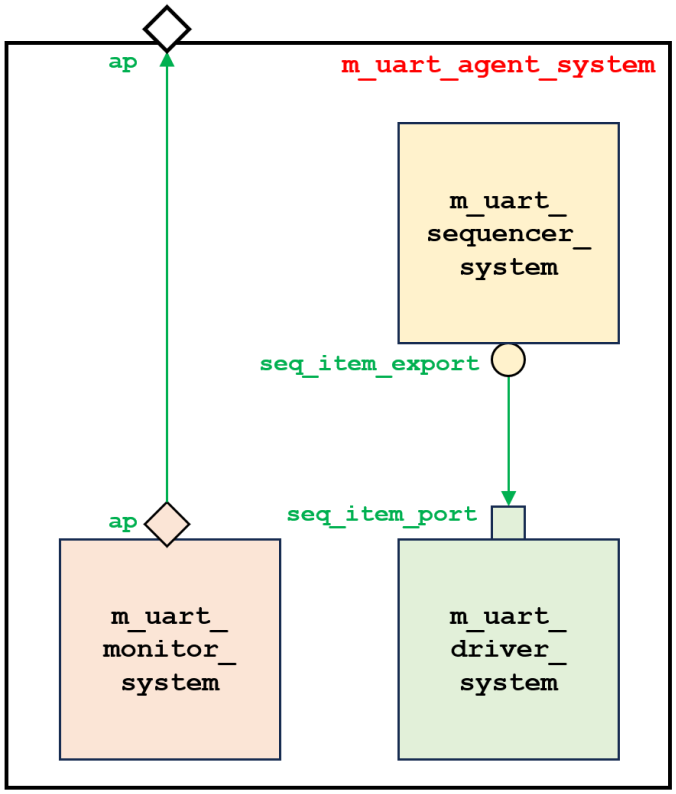

Figure 6 Structure of `m_uart_agent_system`

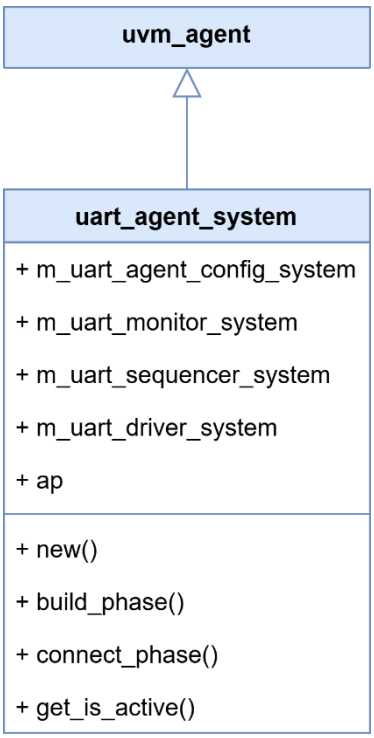

Figure 7 UML class diagram of `uart_agent_system`

4.2 uart_agent_rx

Figure 8 shows the structure of `m_uart_agent_rx`, which is an instance of `uart_agent_rx`.

Figure 9 shows the UML class diagram of `uart_agent_rx`.

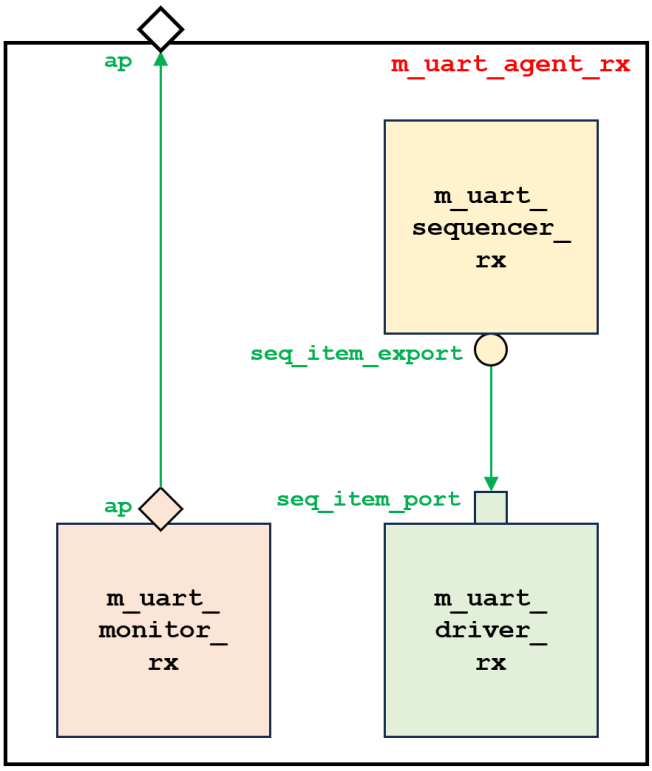

Figure 8 Structure of `m_uart_agent_rx`

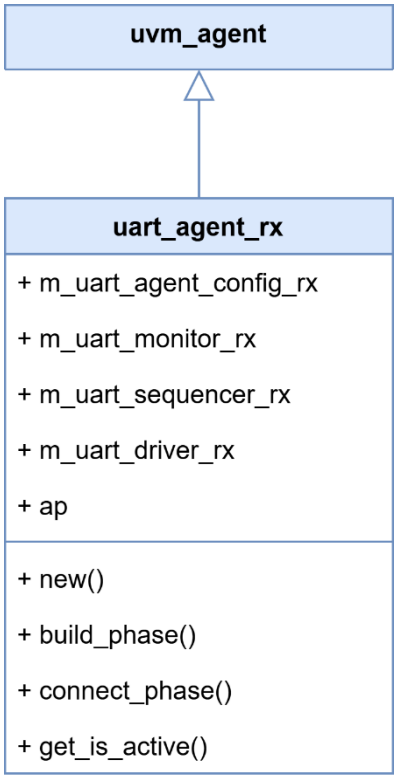

Figure 9 UML class diagram of `uart_agent_rx`

4.3 uart_agent_tx

Figure 10 shows the structure of `m_uart_agent_tx`, which is an instance of `uart_agent_tx`.

Figure 11 shows the UML class diagram of `uart_agent_tx`.

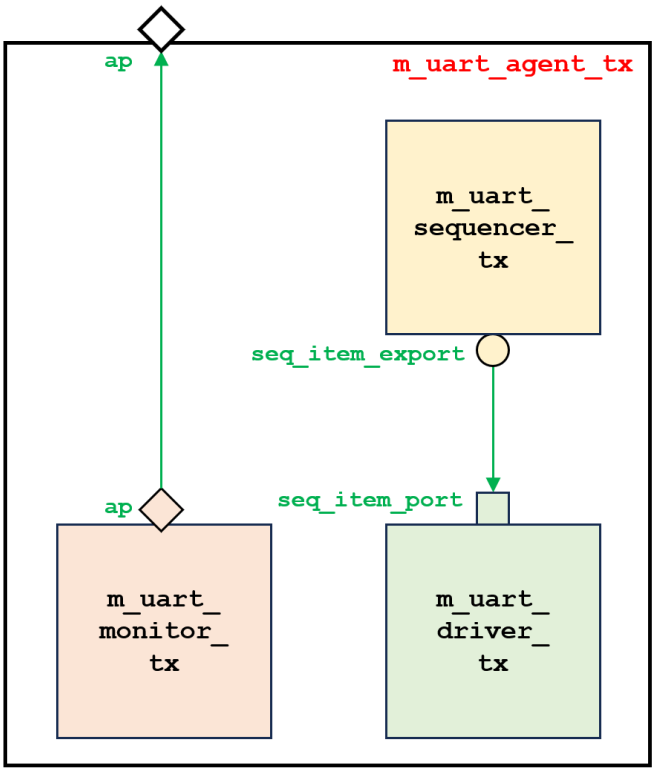

Figure 10 Structure of `m_uart_agent_tx`

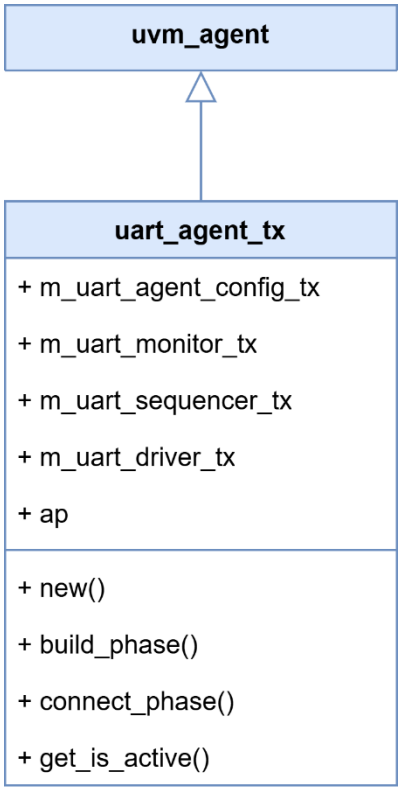

Figure 11 UML class diagram of `uart_agent_tx`

5 Sequence Items

Various sequence items are used in this environment:

- `uart_seq_item_system`
- `uart_seq_item_drv_rx` and `uart_seq_item_mon_rx`
- `uart_seq_item_drv_tx` and `uart_seq_item_mon_tx`

They are described in this chapter.

5.1 System Sequence Item

Figure 12 shows the UML class diagram of `uart_seq_item_system`.

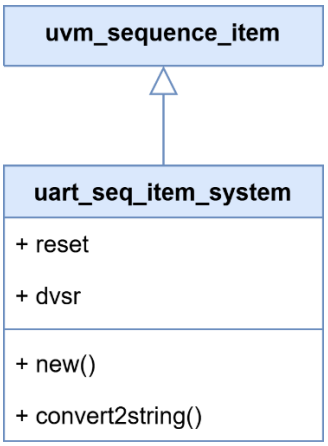

Figure 12 UML class diagram of `uart_seq_item_system`

5.2 Rx Sequence Items

Figure 13 shows the UML class diagrams of `uart_seq_item_*_rx`.

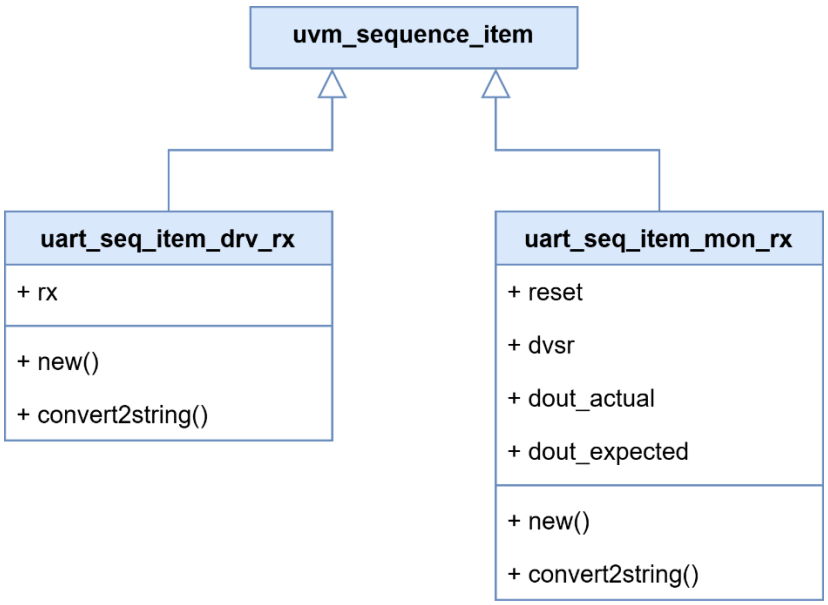

Figure 13 UML class diagrams of `uart_seq_item_*_rx`

5.3 Tx Sequence Items

Figure 14 shows the UML class diagrams of `uart_seq_item_*_tx`.

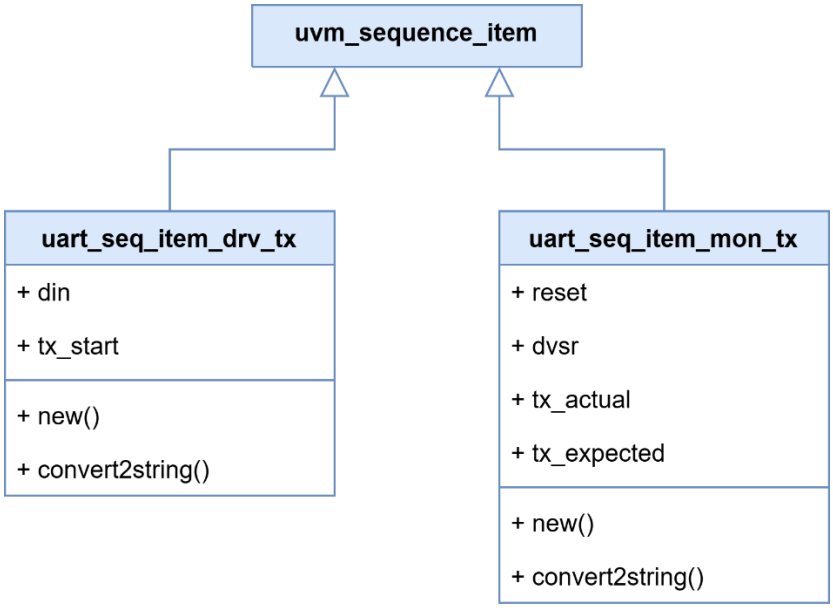

Figure 14 UML class diagrams of `uart_seq_item_*_tx`

6 The Scoreboard

Figure 15 shows the structure of `m_uart_scoreboard`, which is an instance of `uart_scoreboard`.

Figure 16 shows the UML class diagram of `uart_scoreboard` which extends `uvm_scoreboard`.

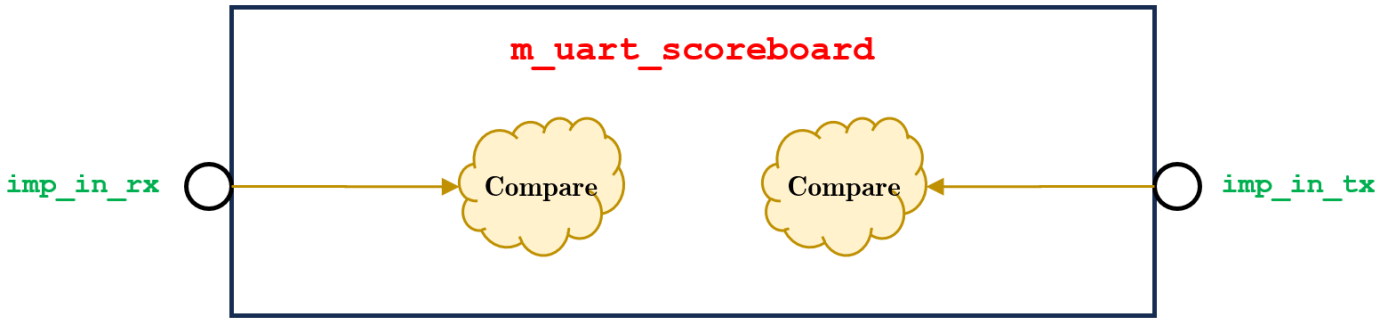

Figure 15 Structure of `m_uart_scoreboard`

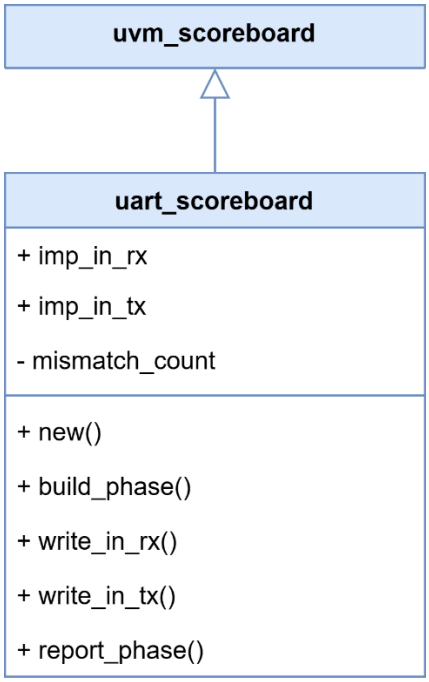

Figure 16 UML class diagram of `uart_scoreboard`

7 Sequences and Tests

Various sequences and tests are used to test the DUT. They are listed in this chapter.[2]

7.1 Sequences

Figure 17 shows the UML class diagrams of `uart_sequence_*` and `uart_virt_sequence_*`.

7.2 Tests

Figure 18 shows the UML class diagrams of `uart_test_*`.

[2] For more details about these sequences and tests, see the test plan in *UART - Verification Plan*.

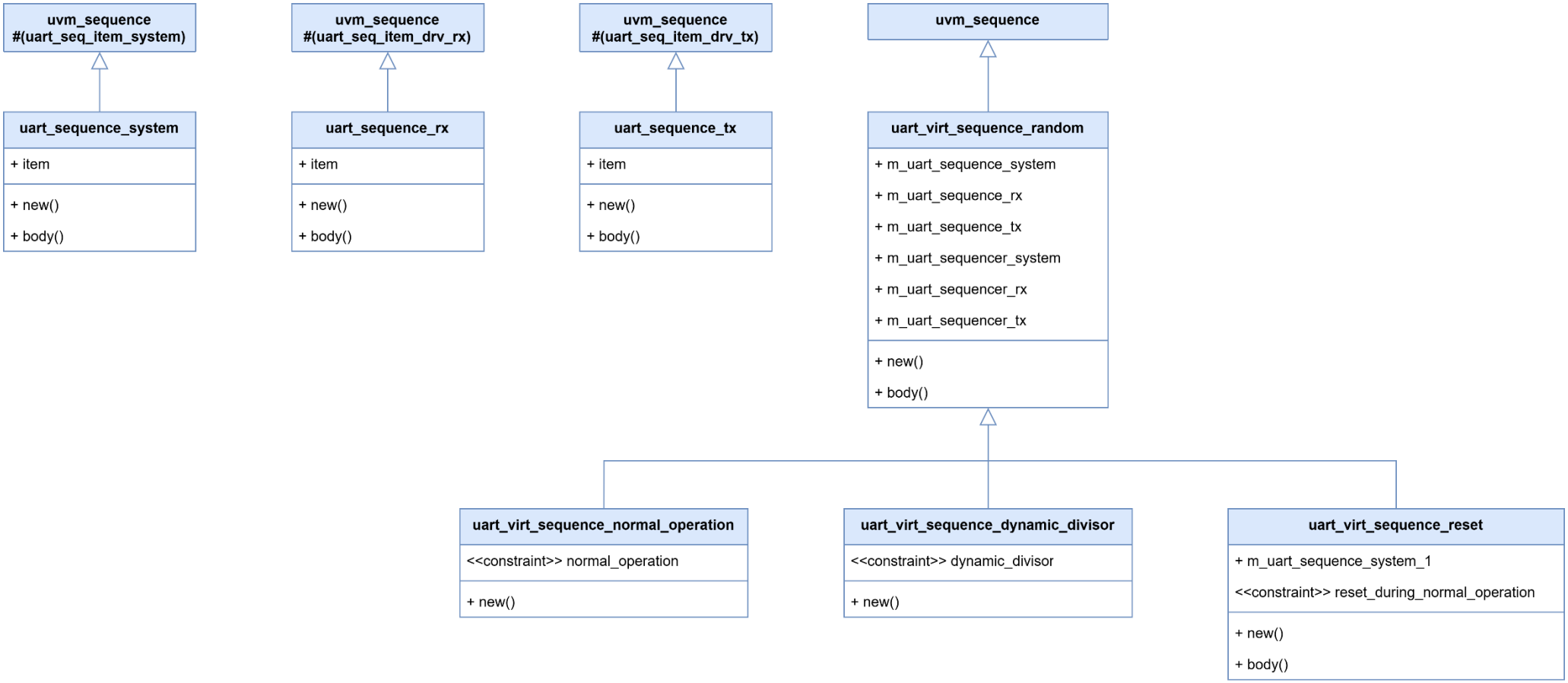

Figure 17 UML class diagrams of `uart_sequence_*` and `uart_virt_sequence_*`

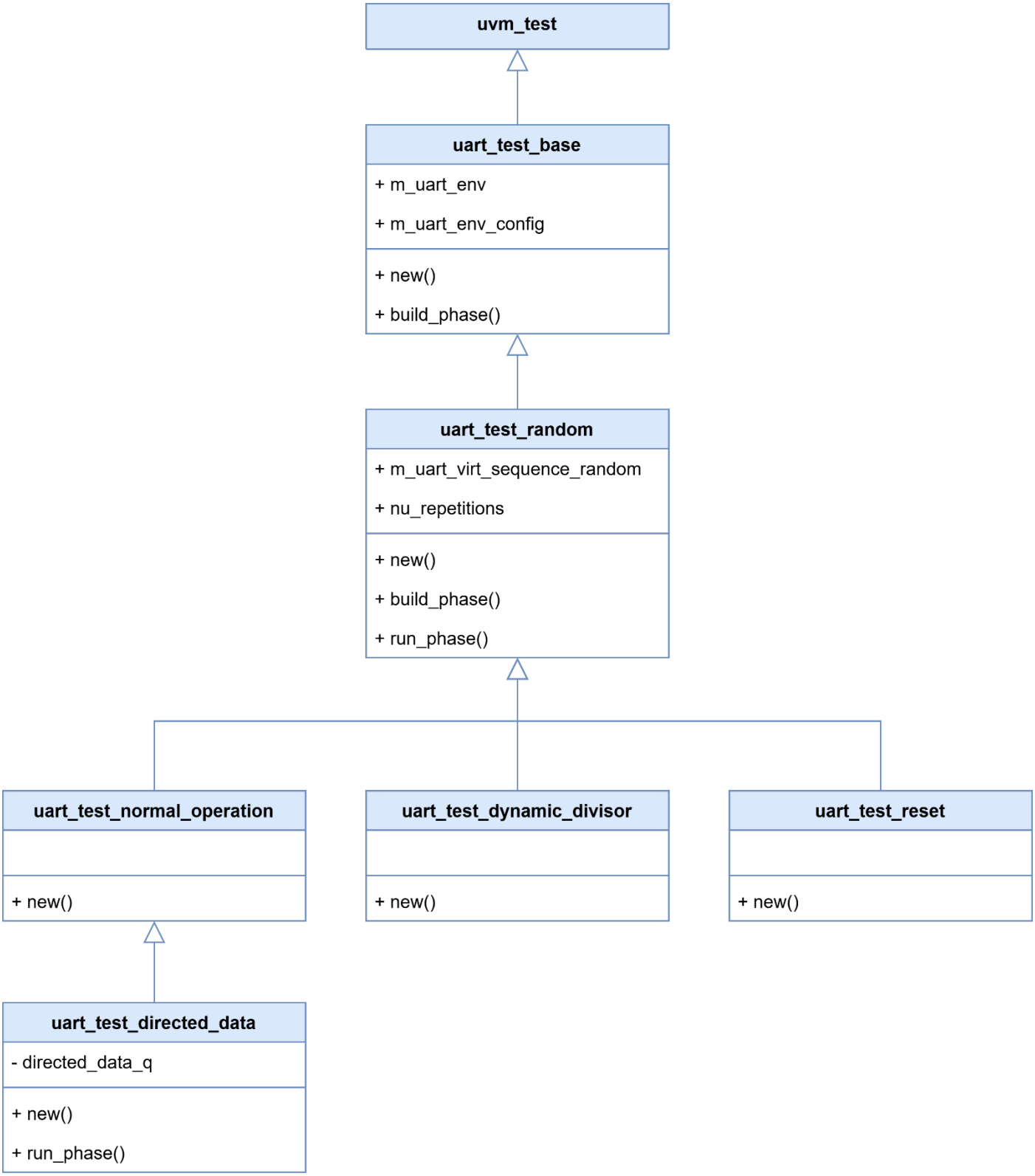

Figure 18 UML class diagrams of `uart_test_*`

Note:

`uart_test_normal_operation`, `uart_test_dynamic_divisor`, and `uart_test_reset`, which extend `uart_test_random`, use the UVM Factory to override `m_uart_virt_sequence_random` with an instance of `uart_virt_sequence_normal_operation`, `uart_virt_sequence_dynamic_divisor`, and `uart_virt_sequence_reset`, respectively.